

Federated Unlearning Made Practical: Seamless Integration via Negated Pseudo-Gradients

Alessio Mora , Carlo Mazzocca , Rebecca Montanari , Paolo Bellavista 

Abstract—The *right to be forgotten* is a fundamental principle of privacy-preserving regulations and extends to Machine Learning (ML) paradigms such as Federated Learning (FL). While FL enhances privacy by enabling collaborative model training without sharing private data, trained models still retain the influence of training data. Federated Unlearning (FU) methods recently proposed often rely on impractical assumptions for real-world FL deployments, such as storing client update histories or requiring access to a publicly available dataset. To address these constraints, this paper introduces a novel method that leverages negated Pseudo-gradients Updates for Federated Unlearning (PUF). Our approach only uses standard client model updates, which are employed during regular FL rounds, and interprets them as pseudo-gradients. When a client needs to be forgotten, we apply the negation of their pseudo-gradients, appropriately scaled, to the global model. Unlike state-of-the-art mechanisms, PUF seamlessly integrates with FL workflows, incurs no additional computational and communication overhead beyond standard FL rounds, and supports concurrent unlearning requests. We extensively evaluated the proposed method on two well-known benchmark image classification datasets (CIFAR-10 and CIFAR-100) and a real-world medical imaging dataset for segmentation (ProstateMRI), using three different neural architectures: two residual networks and a vision transformer. The experimental results across various settings demonstrate that PUF achieves state-of-the-art forgetting effectiveness and recovery time, without relying on any additional assumptions.

Index Terms—Client Unlearning, Federated Learning, Federated Unlearning, Machine Unlearning, Privacy.

I. INTRODUCTION

IN today's digital landscape, privacy has become a major concern, as reflected by the emergence of robust regulatory frameworks worldwide [1]. The European Union (EU) has consistently emphasized the importance of protecting personal data, exemplified by the introduction of the General Data Protection Regulation (GDPR) in 2016 [2]. Most recently, in May 2024, the EU enacted Regulation 2024/1183 [3], establishing the European Digital Identity Framework that empowers individuals with fine-grained control over their information. One of the key rights of these regulations is the *right to be forgotten*, which allows individuals to request the deletion of their previously shared data. Similar rights are central to other major privacy laws worldwide, such as the California Consumer Privacy Act (CCPA) [4] where the *right to delete* grants California residents the on-demand removal of personal data held by businesses.

Alessio Mora, Rebecca Montanari, and Paolo Bellavista are with the Department of Computer Science and Engineering, University of Bologna, Bologna, Italy (e-mail: {name.surname}@unibo.it). Carlo Mazzocca is with the Department of Information and Electrical Engineering and Applied Mathematics, University of Salerno, Fisciano, Italy (email: cmazzocca@unisa.it).

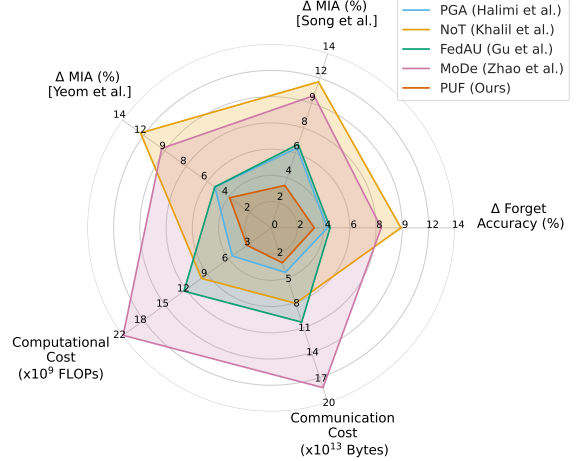


Fig. 1: **Performance comparison of PUF against state-of-the-art mechanisms** using ResNet-18 on CIFAR-100 (non-IID case) in a 10-client setup, where one client requests unlearning. The ideal FU algorithm should (1) minimize the difference with the gold standard *retrained* model, across forgetting metrics such as Forget Accuracy and various MIAs (in Figure expressed as Δ Forget Accuracy and Δ MIAs), (2) minimize computational and communication cost to recover model utility. A smaller polygon represents better unlearning performances. Experimental details are available in Section V, with full results across settings reported in Table III.

Individuals should retain the ability to exercise their privacy rights even in Machine Learning (ML), including withdrawing their contributions from a trained model due to security or privacy concerns [5]. This can be achieved through Machine Unlearning (MU) [6]–[8], an emerging paradigm designed to selectively erase the influence of specific training data from a model by post-processing the trained model.

The *right to be forgotten* should also extend to privacy-preserving ML paradigms such as Federated Learning (FL) [9], [10], which enables the collaborative training of a global model across multiple clients without requiring them to share their private data with a central server. In FL, clients train local models using their on-premises data and only share updates (e.g., weight differences) [11]. Despite offering enhanced privacy, the global model inevitably reflects the influence of client training data [12].

However, the intrinsic characteristics of FL, such as its non-deterministic and iterative training process, make traditional MU techniques less effective in this context [13]. This has led to the novel concept of Federated Unlearning (FU) [13], [14], which encompasses unlearning methods tailored for FL settings. The goal of an FU algorithm depends on what we are trying to remove from the global model, which could be

Work	No Historical Information	No Full Participation	No Stateful Clients	Task Agnostic	No Auxiliary Data	Single-Round Unlearning	Simultaneous Unlearning	Selective Unlearning
Liu et al. [15]	×	×	✓	✓	✓	×	×	✓
FedEraser [16]	×	×	✓	✓	✓	×	×	✓
FUKD [17], [18]	×	✓	✓	×	×	✓	×	✓
PGA [19]	✓	×	×	✓	✓	✓	×	✓
MoDe [20]	✓	✓	✓	×	✓	×	×	✓
FedAU [21]	✓	✓	✓	×	✓	✓	✓	✓
NoT [22]	✓	✓	✓	✓	✓	✓	✓	×
PUF (ours)	✓	✓	✓	✓	✓	✓	✓	✓

TABLE I: Comparison with other works performing client unlearning in federated settings.

either some samples, all the contributions of a client, or all samples belonging to a class. In this work, we address *client unlearning*, where a client seeks to remove all traces of its previously shared data from the global model.

An effective mechanism should achieve unlearning while maintaining the overall performance of the global model. Although some FU techniques have been proposed, most of them rely on hard assumptions as they usually require maintaining additional information, such as historical updates [15]–[17] or stateful clients [19]. Moreover, these works fail to address scenarios where multiple unlearning requests occur in the same time window, a condition that cannot be overlooked, as clients should be allowed to remove their contributions regardless of others’ willingness.

To fill this gap, this paper presents a novel method that leverages negated Pseudo-gradients Updates for Federated Unlearning (PUF). Our approach ensures seamless integration into existing FL frameworks as it enables unlearning without modifying the traditional Federated Averaging (FedAvg) [9] training protocol. Unlike most previous works, PUF eliminates the need for storing historical information (e.g., historical updates), accessing proxy data, maintaining stateful clients, or incurring impractical computational requirements. Since PUF can be seamlessly integrated into standard FedAvg, it also explicitly addresses multiple unlearning requests that concurrently occur. The client who wants to be forgotten sends an unlearning request to the server and performs one final round of FL. The server then adjusts its model updates by flipping and scaling them according to an unlearning rate.

To demonstrate the broad applicability of our approach, we implemented PUF and extensively evaluated its performance on two benchmark image classification datasets (CIFAR-10, CIFAR-100) and a real-world medical imaging dataset (ProstateMRI) for segmentation. The evaluation involved two residual networks and a vision transformer. The reported results clearly show the efficiency and effectiveness of PUF in removing client contributions to the global model across a variety of settings. Figure 1 visually compares the performance of PUF against five state-of-the-art mechanisms. It is worth noting that PUF can also be extended to sample unlearning with minimal modification.

Contributions. The main contributions of this work can be summarized as follows:

- We introduce PUF, a novel approach that enables FU without requiring any modifications or additional overhead to the original FedAvg algorithm.

- We evaluate PUF across different federated settings and compare its performance to two state-of-the-art baselines for client unlearning. We show that PUF is more effective in removing the contribution of the target clients and more efficient in recovering the expected performance.

- We open-source our code to support further related research in the community. The PUF code is available for the research community at: https://github.com/alessiomora/puf_unlearning.

Organization. The remainder of this work is organized as follows. Section III provides the background on FL and FU. Section II reviews the main related work in the field. Section IV presents PUF, while Section V evaluates its performance and Section VI concludes the paper.

II. RELATED WORK

Table I presents a comparative analysis of PUF against existing approaches that target client unlearning. The columns indicate whether each method does not rely on a given requirement (✓) or does depend on it (×). Our comparison highlights key aspects of FU methods, with a primary focus on whether an approach requires and leverages historical updates [15]–[18]. Storing past client model updates raises serious scalability concerns¹. Beyond storage overhead, this requirement introduces a critical limitation: the server must retain the ability to link each update to a specific client, which is fundamentally at odds with the ephemeral and privacy-preserving nature of FL model updates [23]. Furthermore, because these updates must be pre-stored, the data to be removed must be specified in advance, e.g., the entire dataset of a target client. This limitation prevents such methods from supporting more flexible unlearning scenarios, such as when a client requests to remove only a subset of its data after training has already occurred.

In particular, FedEraser [16] uses a multi-round calibration mechanism, which iteratively corrects the historical updates for a set of past retained rounds. However, for each retained round, the remaining clients in the federation must participate in the calibration step; hence, it requires clients to be online and willing to participate in the calibration phase for several

¹To provide a concrete example, consider a ResNet-18 model with approximately 11 million model parameters (around 44 MB per update using 32-bit precision). Retaining updates from 1000 clients over 500 training rounds and across 10 learning tasks would require more than 220 TB of storage. Such a requirement is likely infeasible in practical FL deployments, especially as models, client counts, and training durations continue to grow [10].

rounds. Wu et al. [17], [18] further assume access to auxiliary unlabeled data to mitigate performance degradation due to unlearning. To be effective, such a proxy dataset must be semantically aligned with the data used in the federation and balanced across classes [24]. Furthermore, assuming the availability of a public dataset and relying on it for unlearning in FL is often unrealistic in practice [25].

Given these limitations, we primarily focus our comparison on more practical FU methods. Halimi et al. [19] propose Project Gradient Ascent (PGA) to erase client influence by constraining local weight updates within an L2-norm ball around a reference model. While PGA does not require historical updates or auxiliary data, it introduces other constraints. Clients must be stateful, storing their latest model updates, and full participation is mandatory, as unlearning relies on a reference model obtained by removing the last target client's update from the current global model. Additionally, directly applying gradient ascent on forgot data risks gradient explosion, as the loss function often lacks an upper bound, leading to divergence and poor generalization. To counteract this, PGA employs gradient clipping and L2-norm constraints [19]. However, gradient-ascent techniques require careful and extensive hyperparameter tuning, such as selecting the appropriate L2-norm radius or gradient clipping threshold. PUF eliminates the risk of uncontrolled gradient growth while maintaining the standard local training procedure. This is achieved by rescaling the client's local update (pseudo-gradient) after training, rather than modifying the training dynamics.

MoDe [20] introduces a multi-round unlearning protocol alternating between degradation and memory-guidance phases. This requires additional per-round coordination among clients with respect to FedAvg, as well as more per-round computational and communication requirements. In contrast, PUF does not require any change in the regular FedAvg training round. FedAU [21] trains a local auxiliary module on forget data with randomized labels, and forms the unlearned model as a linear combination of the original model and the auxiliary module. This mechanism requires clients to train an auxiliary module during the regular FedAvg rounds, hence introducing a variation over the base protocol. Similarly, FedUNRAN [26] fine-tunes the global model on forget data with randomized labels. Recently, Khalil et al. [22] proposed NoT, a novel method aiming to achieve unlearning without impractical assumptions by implementing server-side unlearning. Specifically, NoT inverts the sign of the first layer's global model parameters upon receiving an unlearning request. While this approach does not require explicit client-side unlearning, it fails to remove a target client's contribution selectively. In fact, NoT produces the same unlearned model regardless of which client exits the federation. Our experimental evaluation (see Figure 1 and Section V) shows that NoT requires a longer recovery period and fails to match the forgetting performance of full model retraining.

In contrast to previous methods, PUF offers several key advantages. It does not rely on historical updates or auxiliary data, does not require full client participation, and operates in a stateless manner. Moreover, PUF is task-agnostic, meaning it does not need modifications based on the specific learning

task. It maintains the same computational cost as standard FedAvg rounds and requires minimal tuning, as demonstrated in our experimental results. Furthermore, PUF supports sample unlearning and can handle multiple unlearning requests simultaneously. These properties make PUF a practical and effective solution for federated unlearning, inherently aligned with the design principles of FL.

III. BACKGROUND

A. Federated Learning

FedAvg [9] is a widely used baseline in FL. It adopts a client-server framework in which a central server manages the global model parameters. Training proceeds in synchronous rounds, where a randomly selected subset of client devices is invited to participate. During each round, the activated clients fine-tune the global model using their local data over a fixed number of epochs. The server then collects the locally computed updates from the clients, and incorporates them into the global model w , before distributing the updated global parameters again. In FedAvg, clients typically send model updates to the server. These updates are calculated as the difference between the global weights, received at the beginning of the round, and the locally trained weights. This can be expressed mathematically as:

$$w_{t+1} = \frac{1}{|S_t|} \sum_{i \in S_t} w_t^i = w_t + \frac{1}{|S_t|} \sum_{i \in S_t} (w_t^i - w_t) \quad (1)$$

where S_t represents the set of clients selected for round t , w represents the weight of the global model and w^i are the weights computed locally at client i . For clarity, in Equation 1, model updates are shown without being scaled by the number of local data samples, but this simplification does not affect the subsequent analysis. By defining client updates as $\Delta_t^i := w_t^i - w_t$ and their aggregated form as:

$$\Delta_t := \frac{1}{|S_t|} \sum_{i \in S_t} \Delta_t^i, \quad (2)$$

and by introducing a (global) learning rate η_s , the FedAvg's update rule can be rewritten as:

$$w_{t+1} = w_t + \eta_s \Delta_t = w_t - \eta_s (-\Delta_t) \quad (3)$$

This shows that the server's update rule in FedAvg is equivalent to applying SGD to the *pseudo-gradient* $-\Delta_t$ with $\eta_s = 1$. This perspective reveals that FedAvg is a specific instance of the broader FedOpt framework [27], which can use various server-side optimizers. Building on this view, Reddi et al. [27] formally treat $-\Delta_t$ as a stochastic gradient in their FedOpt framework and prove convergence guarantees for a wide range of server optimizers (e.g., SGD, Adam, Yogi) under standard smoothness and bounded-variance assumptions. This establishes that the aggregated client update has the same mathematical role as a true gradient in the server's optimization step, despite the server never accessing raw data.

B. Federated Unlearning

An FU method aims to remove specific learned information from the global model w while preserving the *good* knowledge acquired on data that should not be forgotten. FU objectives are classified based on the information the algorithm is expected to forget: sample unlearning, class unlearning [28], feature unlearning [29], and client unlearning. Our work mainly focuses on client unlearning.

During a training round t , a client u may withdraw previous contributions from the global model w . This is achieved through an unlearning procedure $\mathcal{U}(w_t, D_u)$, which can be executed by different entities within the FL framework, such as the server, the client to be forgotten, or the remaining clients [30]. An unlearning algorithm should produce a novel version of the original global model w_t^u , called the unlearned model, that effectively excludes the influence of the forget data D_u . The method is effective if w_t^u exhibits performances approximately indistinguishable from the retrained model w_t^r , a model trained as if client u had never joined the federation.

The efficiency of FU is usually measured by the number of rounds needed to obtain the generalization performance of w_t^r . Effectiveness in forgetting the client data D_u is assessed by comparing the performance of w_t^u and w_t^r on D_u . Common metrics for verifying the success of unlearning include loss, accuracy, and susceptibility to membership inference attacks (MIAs) [31]–[33]. MIAs test whether certain samples were used at training time, by providing a measure of the unlearning effectiveness in removing traces of such samples.

C. Terminology

In this subsection, we define key terminology that will be used throughout the paper.

- **Target client:** The client that requests unlearning, also referred to as the *Target Client* or client u . The objective of an unlearning algorithm is to remove the influence of client u data (i.e., forget data) from the global model.
- **Original Model:** The global model trained with the participation of both the target client and all other clients.
- **Retrained Model:** The global model trained without the target client from the beginning of the FL process. This serves as the gold-standard baseline for evaluating the effectiveness of unlearning methods.
- **Unlearned Model:** The original model after the unlearning procedure (for example, after applying PUF). Since unlearning may impact model performance, a recovery phase is often necessary to restore its generalization capabilities to match the retrained model. As detailed in Section V, an optimal FU algorithm should minimize the discrepancy with the retrained model in terms of forgetting metrics (e.g., accuracy on client u 's data, susceptibility to MIAs) while achieving equal or superior generalization to the retrained model in the fewest possible FL rounds.

IV. PUF: FEDERATED UNLEARNING VIA NEGATED PSEUDO-GRADIENTS

A. Preliminaries

Before presenting our original method, we provide the necessary preliminaries and notation to fully and more easily understand our proposal. During a given round t , a subset of the participating clients may request the removal of their contributions. We define S_t^- as the subset of clients requesting unlearning, and S_t^+ as the remaining clients. Additionally, let n denote the total number of samples held by participating clients during the unlearning round, such that:

$$n := \sum_{i \in S_t^+} |D_i| + \sum_{j \in S_t^-} |D_j| \quad (4)$$

We define two aggregated model updates: Δ_t^+ , representing the aggregated updates contributed by the clients in S_t^+ , and Δ_t^- , representing the aggregated updates contributed by the clients in S_t^- , as follows:

$$\Delta_t^+ := \frac{1}{n} \sum_{i \in S_t^+} \Delta_t^i = \frac{1}{n} \sum_{i \in S_t^+} |D_i| (w_t^i - w_t) \quad (5)$$

$$\Delta_t^- := \frac{1}{n} \sum_{j \in S_t^-} \Delta_t^j = \frac{1}{n} \sum_{j \in S_t^-} |D_j| (w_t^j - w_t) \quad (6)$$

We also define the forget data (or target data) as the union of the local dataset held by clients that request unlearning, expressed as $D_u := \bigcup_{j \in S_t^-} D_j$.

B. Federated Unlearning via Negated Pseudo-Gradients

Motivation. Existing active FU approaches [16], [19], [21], [34] require non-trivial modifications to the standard FedAvg protocol. They may introduce multiple rounds of specialized aggregation, alter the local training procedure, or rely on auxiliary modules, auxiliary data, and modified loss functions.

In contrast, we aim for a *minimally disruptive* unlearning mechanism that integrates seamlessly into the existing training pipeline. To this end, interpreting negated pseudo-gradients as a form of gradient ascent on the forget data achieves it naturally: it preserves the standard update interface of FedAvg, requires no changes to the aggregation rule, and introduces no additional objectives or model components at the client side. In the following, we provide further insights into PUF's design choice.

Intuition. In FedAvg [9], clients receive the weights of the current global model and send back model updates. As discussed in Section III, these updates can be interpreted as pseudo-gradients, also enabling the use of server-side optimizers [27]. This interpretation is not merely conceptual: in the FedOpt framework of Reddi et al. [27], the server update in FedAvg is mathematically equivalent to applying a gradient-based optimizer to the pseudo-gradient $-\Delta_t$, with formal convergence guarantees under standard assumptions. We leverage this interpretation for unlearning in PUF. Specifically, PUF flips the sign of the model update (pseudo-gradient) computed by the target client(s), applying it in the opposite direction of the

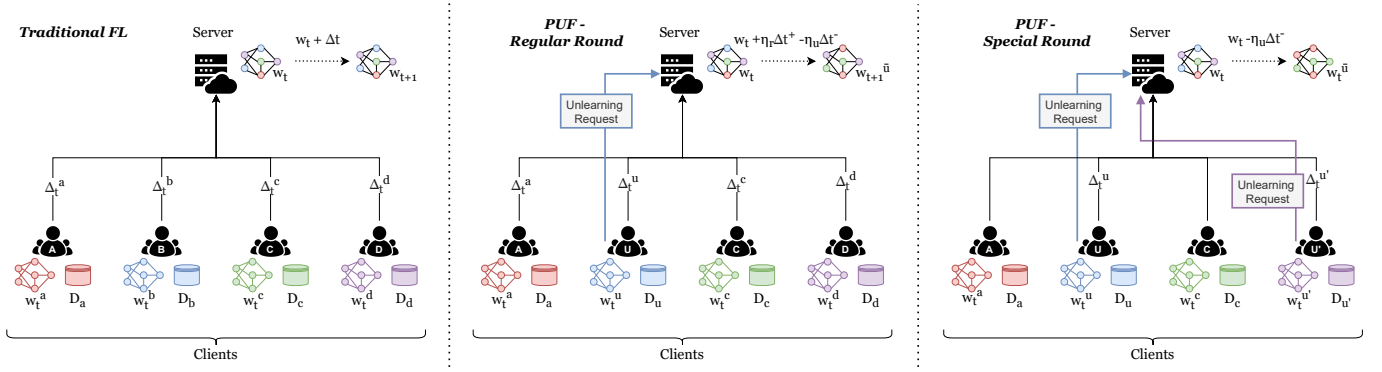


Fig. 2: Overview of PUF operating modes. In PUF-Regular Round, target clients provide their model updates together with other clients. In PUF-Special Round, only target clients share their model updates.

local optimization. Figure 2 provides an illustrative overview of our method.

This approach emulates the effect of gradient ascent-based unlearning techniques by pushing the global model parameters away from regions where the loss function on the forget data is minimized. A key technical advantage of PUF over directly applying gradient ascent, which maximizes the loss function rather than minimizing it on the forget data, is that it avoids the risk of gradient explosion. Since the loss function typically lacks an upper bound, gradient-ascent techniques can lead to unbounded gradient growth, preventing convergence and severely degrading the model’s generalization performance. To address this, regularization methods such as weight projection (e.g., L2-norm ball) and gradient clipping are often employed to constrain parameter updates. However, these techniques require careful, often extensive hyperparameter tuning, including selecting an appropriate L2-norm radius or a gradient clipping threshold.

Our Proposal. PUF eliminates the risk of uncontrolled gradient growth by preserving the standard local training procedure on clients while negating and rescaling the resulting update (pseudo-gradient) after local training. Our unlearning method supports two alternative operating modes:

- **Regular round with modified aggregation (PUF-Regular).** In this mode, unlearning is integrated into a regular training round, where both unlearning and remaining clients participate. When one or more clients opt to leave the federation and be forgotten, the round proceeds as usual, except that the aggregated model updates from target clients are negated. The sign flip can be applied either locally at clients, before transmitting back the updates, or during the server-side aggregation. The updates from other clients remain unchanged. The resulting aggregation can be formalized as follows:

$$\Delta_t = \eta_r \Delta_t^+ - \eta_u \Delta_t^-, \quad (7)$$

with η_r and η_u being global learning rates. In particular, η_u scales the aggregated update from target clients. Considering SGD as a server-side optimizer and $\eta_s = 1$ (see Eq. 3), the resulting unlearning model is generated as:

$$w_{\bar{u}} = w_t + \Delta_t \quad (8)$$

Regular FL training resumes from $w_{\bar{u}}$.

- **Special unlearning round with only target clients (PUF-Special).** In this mode, a dedicated unlearning round is conducted before resuming regular training. Only the target clients participate, retrieving the latest global model, and performing regular training. Assuming SGD as the server-side optimizer with $\eta_s = 1$, the aggregated update is negated and applied to the global model as follows:

$$w_{\bar{u}} = w_t - \eta_u \Delta_t^-, \quad (9)$$

where η_u is the scaling factor for the aggregated update from target clients. Regular FL training resumes from $w_{\bar{u}}$.

Algorithm 1 reports a unified framework for both PUF-Special and PUF-Regular. For readability, at line 14, we assume SGD with $\eta_s=1.0$ as the server-side optimizer. It is important to emphasize that PUF-Regular and PUF-Special are *alternative* unlearning modes, and are not meant to be used jointly or sequentially. For any given unlearning request, only one of the two routines should be applied.

We included both variants, PUF-Regular and PUF-Special, to support different system constraints. PUF-Regular is designed to seamlessly operate in standard FL workflows, where both target and regular clients participate in the same round. This makes PUF-Regular fully transparent to the federation and particularly suitable for real-world deployments where interrupting the training process or coordinating a dedicated round may not be feasible. In contrast, PUF-Special reflects a more conventional design choice, consistent with prior works (e.g., [19]), where unlearning is isolated into a dedicated round involving only the target clients. While this enables a more controlled and potentially fine-grained unlearning process, it comes at the cost of requiring explicit coordination.

C. Discussion

Threat Model. Since the server performs unlearning, our threat model adopts the widely accepted *honest-but-curious* server assumption, consistent with prior FU literature [13],

Algorithm 1 PUF Algorithm. Note that $S_t^+ = \emptyset$ when PUF-Special is used for the unlearning phase, while $S_t^+ \neq \emptyset$ and $S_t^- \neq \emptyset$ for PUF-Regular. $S_t^- = \emptyset$ for regular training rounds.

Input: Global model weights w , local epochs E , global learning rate η_s , unlearning rate η_u , local learning rate η , batch size B

- 1: Initialize w_0
- 2: **for** each round $t = 0, 1, 2, \dots$ **do**
- 3: **if** t is unlearning round **then**
- 4: $S_t^+ \leftarrow$ (random set of remaining clients)
- 5: $\triangleright S_t^+ = \emptyset$ if PUF-Special
- 6: $S_t^- \leftarrow$ (set of target clients)
- 7: **else** $\triangleright$ Standard FedAvg Round
- 8: $S_t^+ \leftarrow$ (random set of clients)
- 9: $S_t^- = \emptyset$
- 10: **for** each client $i \in S_t^+$ **simultaneously do**
- 11: $\Delta_t^i \leftarrow \text{ClientOpt}(i, w_t)$
- 12: **for** each client $j \in S_t^-$ **simultaneously do**
- 13: $\Delta_t^j \leftarrow \text{ClientOpt}(j, w_t)$
- 14: $\Delta_t^+ \leftarrow \frac{1}{n} \sum_{i \in S_t^+} \Delta_t^i$
- 15: $\Delta_t^- \leftarrow \frac{1}{n} \sum_{j \in S_t^-} \Delta_t^j$
- 16: $\Delta_t \leftarrow \eta_r \Delta_t^+ - \eta_u \Delta_t^-$
- 17: $w_{t+1} \leftarrow w_t + \Delta_t$
- 18: **procedure** CLIENTOPT(k, w_t)
- 19: $w \leftarrow w_t$
- 20: $\mathcal{B} \leftarrow$ (split $\mathcal{D}_k$ into batches of size B)
- 21: **for** each local epoch e from 1 to E **do**
- 22: **for** each batch $b \in \mathcal{B}$ **do**
- 23: $w \leftarrow w - \eta \nabla \ell(w; b)$
- 24: $\Delta_t^k \leftarrow w - w_t$
- 25: **return** Δ_t^k to server

[35]. Although negated pseudo-gradients could, in principle, be vulnerable to adversarial manipulation, PUF does not introduce new vulnerabilities beyond those already present in standard FL. In particular, the use of negated pseudo-gradients follows the same abstraction as standard client updates in FedAvg, and is therefore subject to similar adversarial risks.

Due to its seamless integration into the FedAvg pipeline, PUF is naturally compatible with existing Byzantine-robust aggregation and filtering mechanisms (e.g., Krum, Multi-Krum [36], or Median-based [37] approaches). In practice, client updates can be first collected in their original form (i.e., without sign inversion or scaling) and processed through a robust aggregation/filtering step to select a subset of consistent updates. The PUF transformation (i.e., sign inversion and scaling) can then be applied only to the selected updates before aggregation. This modular design preserves the robustness properties of the underlying aggregation scheme while enabling unlearning, without requiring modifications to the protocol.

Extension to Sample Unlearning. Our work mainly focuses on client unlearning. However, PUF can be readily extended to sample unlearning scenarios. In this setting, during the unlearning round, the clients that request to be forgotten use

only the subset of their local dataset that must be removed when computing the model update.

Impact of Scaling Factor η_u . The effectiveness of PUF depends on the magnitude of the unlearning updates, which must be carefully controlled to prevent excessive model drift. As we will discuss in Section V, the tuning of the scaling factor η_u is crucial for balancing unlearning effectiveness while mitigating degradation in the model’s generalization performance. PUF efficiently removes contributions from target clients while remaining fully compatible with the traditional FedAvg. After the unlearning round, the global model may require a few additional rounds to recover its generalization performance, as observed in all approximate FU mechanisms [13]. It is worth noting that in both operating modes, clients can transmit model updates to the server, as done in traditional FedAvg, without requiring any modifications or additional overhead. PUF is designed to ensure the unlearning process is task-agnostic and seamless for federated participants, supporting multiple concurrent unlearning requests.

Resilience to Client Dropouts. PUF-Special and PUF-Regular can be interpreted as two operational extremes: PUF-Special involves only the target client in a dedicated round, while PUF-Regular assumes standard participation of multiple clients. In intermediate scenarios where some clients drop out during the unlearning round, PUF-Regular can still be applied without structural changes. The main effect is a change in the relative contribution of the target client during aggregation; in practice, this can be compensated by adjusting the scaling factor η_u . A practical approach is to scale the PUF-Regular η_u proportionally to the fraction of active clients, i.e., accounting for the number of client dropouts in the round.

D. Limitation

PUF assumes that the target clients participate one final time in the unlearning phase to generate the pseudo-gradient update from the forget data. While this assumption is common in prior FU methods [19]–[21], it represents a practical limitation in scenarios where the requesting clients are unavailable, unresponsive, or unwilling to participate at the time of the unlearning request. Furthermore, due to the stateless and ephemeral nature of PUF, which is designed to remain fully aligned with standard FedAvg, unlearning updates are computed solely with respect to the current global model. On the one hand, this design avoids relying on historical information and aligns well with data minimization principles; on the other hand, it reinforces the requirement for the active participation of the requesting clients during unlearning.

V. EXPERIMENTAL RESULTS

This section presents a comprehensive evaluation of PUF across a diverse set of configurations. Table II provides an overview of the experimental settings, which are discussed in detail in the following subsections.

A. Datasets, Models, and Learning Setting

Datasets. We conducted a comprehensive set of experiments on image classification tasks and image segmentation tasks.

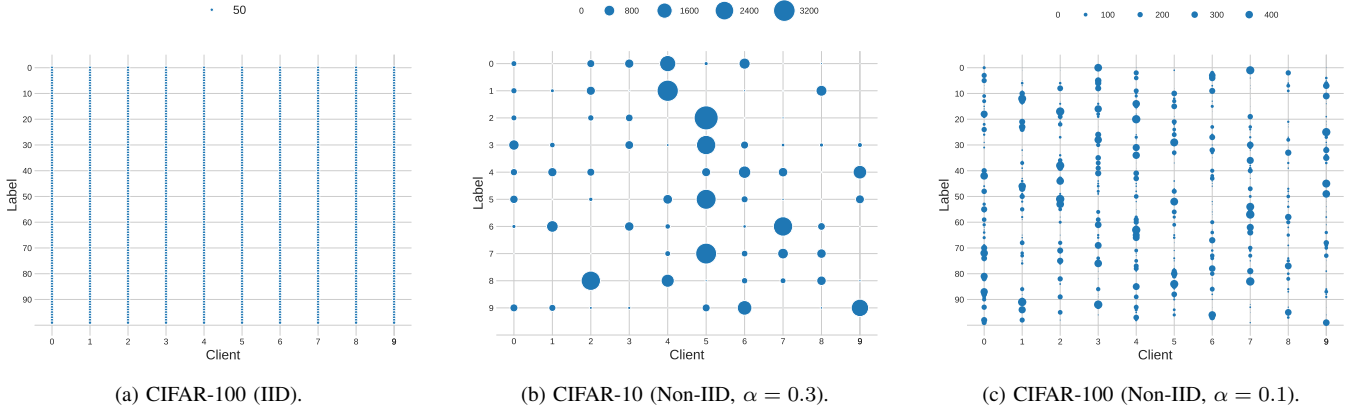


Fig. 3: Label distribution across clients (0-9) for CIFAR-100 (IID), CIFAR-10 (Non-IID, $\alpha = 0.3$), and CIFAR-100 (Non-IID, $\alpha = 0.1$).

Model	Dataset	Clients	Unlearning Case	Data Heterogeneity	Pre-trained	E	Task	# target clients
ResNet-18	CIFAR-10	10	Client	LDA ($\alpha=0.3$)	×	1	$\mathcal{C}$	Up to one
ResNet-18	CIFAR-100	10	Client, Backdoor	LDA ($\alpha=0.1$) or IID	×	1 or 10	$\mathcal{C}$	Up to two
ResNet-18	CIFAR-100	10	Sample	LDA ($\alpha=0.1$) or IID	×	1 or 10	$\mathcal{C}$	Up to one
MiT-B0	CIFAR-100	10	Client	LDA ($\alpha=0.1$)	✓	1	$\mathcal{C}$	Up to two
ResUNET	ProstateMRI	6	Client	Real-world feature skew	×	1	$\mathcal{S}$	Up to one

TABLE II: Settings of reported results. $\mathcal{C}$ =classification task, $\mathcal{S}$ =segmentation task. E =local epochs during FedAvg. # target clients reports if the experiments consider a single client or multiple clients to unlearn.

For image classification, we used federated versions of the CIFAR-10 and CIFAR-100 datasets [38], which consist of 60,000 32×32 color images. CIFAR-10 comprises 10 classes, while CIFAR-100 includes 100 classes. The datasets were partitioned to simulate 10 clients, ensuring no overlapping samples among them. The experiments were performed under both Identically and Independently Distributed (IID) and non-IID settings across clients. To create non-IID data, we applied label-skew partitioning based on a distribution determined by Latent Dirichlet Allocation (LDA) [39], using concentration parameters of $\alpha = 0.3$ for CIFAR-10 and $\alpha = 0.1$ for CIFAR-100. Figure 3 provides a visual representation of the label distribution among clients.

For image segmentation, we conducted a set of experiments using the ProstateMRI federated dataset [40] that comprises prostate T2-weighted MRI scans (with segmentation masks) sourced from six data providers, each one treated as a client. Due to variations in imaging protocols across sites, real-world feature heterogeneity naturally arises among clients. The dataset includes 384×384 color images alongside their corresponding segmentation masks.

Model architectures. For image classification, we used a standard ResNet-18 [41] and a visual transformer, namely the MiT-B0 [42]. For image segmentation, we used a vanilla ResUNet architecture [43], similar to the one used in [40].

Unlearning Objectives. We provide results for client unlearning, sample unlearning, and for simultaneous unlearning of multiple clients. For sample unlearning, we used the same configurations and hyperparameters as in the client unlearning setup. When evaluating sample unlearning, 50% of the samples in the local dataset of each client that requests to be forgotten are removed. The forget subset is obtained through uniform random selection from the client’s local dataset. The remaining

samples are then used in the recovery phase for all considered methods, meaning that the target clients contribute to the recovery process only with their retain data.

Backdoor Injection. For CIFAR-100 experiment under IID data, we also evaluate the performance for backdoor removal, following the experimental protocol of [19]. Training is first performed in a standard FL setting over all clients, where a designated forget client is poisoned: a fixed fraction of its local samples, excluding those already belonging to the target class, is modified by inserting a deterministic 3×3 trigger in a fixed corner of the image and relabeling them to a chosen target class. We adopt a simple pixel-pattern trigger (constant across samples), but we adapt it to our preprocessing pipeline by injecting it after data augmentation and normalization.

Learning Setting. In all experiments, we considered SGD as a server-side optimizer with $\eta_s=1.0$, and we set $\eta_r=1.0$ for PUF-Regular. If not differently indicated, we set $\eta_u=2.0$ for PUF-Special and $\eta_u=20.0$ for PUF-Regular (Section V-E discusses the tuning of our methods). Further details about per-setting hyper-parameters are reported in the following.

ResNet-18 on CIFAR-10/CIFAR-100. We used a standard ResNet-18 [41] and employed Group Normalization layers, similar to other works with similar settings (e.g., [44]). We used SGD as a local optimizer with a learning rate set to 0.1, with a round-wise exponential decay of 0.998, 1 or 10 local epochs ($E = 1$ or $E = 10$), local batch size of 32. We pre-processed the training images with random crop, horizontal flip and normalization layers. Before performing unlearning, we train ResNet-18 from scratch for 200 FL rounds while. If not differently indicated, clients run one local epoch ($E = 1$) for standard FedAvg. During unlearning routines, we only apply normalization.

MiT-B0 on CIFAR-100. We used a visual transformer, i.e.,

MiT-B0 [42], with approximately 3.6M parameters, initialized from a pre-trained model checkpoint trained on ImageNet-1k (69.27% accuracy on test data). Although originally proposed for segmentation, MiT-B0 features a task-agnostic backbone and is adapted to classification by replacing its task-specific head, similarly to standard fine-tuning procedures. Accordingly, we replaced the original head with a one-layer classification head, initialized from scratch. We employed the AdamW optimizer with a client learning rate of $3e-4$, with a round-wise exponential decay of 0.998, 1 local epoch, local batch size of 32, and weight decay regularization of $1e-3$. The images are resized to a resolution of 224×224 . Before performing unlearning, we fine-tune MiT-B0 for 50 FL rounds. If not differently indicated, clients run one local epoch ($E = 1$) for standard FedAvg.

Res-UNet on ProstateMRI. In line with [45], we trained the vanilla Res-UNet using a combination of standard cross-entropy and Dice loss [46], and we conducted regular training for 500 rounds, before applying unlearning. We used Adam [47] as local optimizer with a client learning rate of $1e-4$, a local batch size of 16, and a local weight decay of $1e-4$.

B. Baselines

We compared PUF against five state-of-the-art baseline methods, which are FedEraser [16], PGA [19], FedAU [21], MoDe [20] and NoT [22]. Our comparisons primarily focus on FU methods that avoid impractical assumptions, such as the need for historical information. Nevertheless, we also include FedEraser [16], despite its requirement to store past client models, as it serves both as a widely adopted reference baseline in the literature and as a means to illustrate the overhead incurred by methods that rely on this requirement. Hyper-parameter tuning of baseline methods can be found in Appendix A (Supplementary Material).

As PGA [19] and NoT [22] were originally evaluated with full client participation, both during the training of the original model and in the recovery phase, we adopted the same setting to ensure a fair comparison.

In some experiments, we included a *Natural* baseline, which bypasses any unlearning procedure before the recovery phase. This allows us to assess whether the influence of the target client would naturally vanish over time without intervention.

C. Metrics

We evaluated the proposed methods along two key dimensions: their *efficacy* in achieving unlearning and their *efficiency* in recovering model utility. For unlearning efficacy, the primary goal is to minimize the discrepancy between the metrics of the approximate unlearning method and those of the gold-standard retrained model. In this context, neither higher nor lower absolute metric values are inherently preferable; instead, the objective is to reduce the absolute difference with respect to retraining from scratch. We denote this absolute difference with the prefix Δ (e.g., Δ Test Accuracy represents the absolute difference in Test Accuracy between the unlearned model and the retrained model). For unlearning efficiency, we

measure the cumulative *computational* (FLOPs), *communication* (bytes), and *storage* (bytes) overhead required to perform unlearning and to recover model utility. In addition, we report the *relative improvement* over retraining from scratch, where higher gains indicate better performance.

Test Accuracy (Δ Test Acc.). We evaluated the test accuracy at two critical stages: immediately after the unlearning routine (before the recovery phase) and after the recovery phase concludes. We used the standard test set provided with the dataset, which is IID and contains data that have never been accessed or observed by any client during training.

Forget Accuracy (Δ Forget Acc.). We evaluated the unlearned model's accuracy on client u 's train data and computed the absolute difference relative to the retrained model's performance. This metric provides insight into how closely the unlearning process approximates the desired outcome achieved through retraining.

MIAs on Forget Data (Δ MIA). The MIA aims to infer whether specific samples were part of the training data for the attacked model. Its success rate quantifies the fraction of target data, defined as $(D_u := \bigcup_{j \in S_u^-} D_j)$, correctly identified as belonging to the training set. A lower MIA success rate indicates reduced information leakage about D_u from the attacked model. To implement MIAs, we employed two established approaches: (i) a confidence-based MIA predictor [48], and (ii) a loss-based MIA predictor [49].

- 1) Confidence-based MIA predictor [48]: It consists of a training phase using a balanced dataset of seen and unseen data. The retain dataset serves as the seen data, while the standard test dataset is used as the unseen data. An MIA predictor is then trained to classify whether the target model's output corresponds to seen or unseen data based on prediction confidence.
- 2) Loss-based MIA predictor [49]: It assumes the attacker can access the target model's average training loss. Samples are classified as members of the training set if their loss falls below this average; otherwise, they are labeled as non-members. For our federated implementation, we assume access to the global mean training loss.

Backdoor Attack Success Rate. We evaluate accuracy on a backdoor test set following [19]. Given a target class, the backdoor test set is constructed from the standard test set by removing all samples that already belong to the target class. The trigger (see *Backdoor Injection* in Section V.A) is then applied to the remaining samples, which are relabeled with the target class. The accuracy on this dataset corresponds to the Attack Success Rate (ASR), i.e., the probability that the model predicts the attacker's target label in the presence of the trigger. A high ASR indicates a successful backdoor attack.

Communication, Computational, and Storage Costs. We compute cumulative communication, computational, and storage costs for each FU method, considering both the unlearning phase and the recovery phase. In the unlearning phase, we quantify these costs according to the original design of each algorithm, under consistent assumptions between methods. The recovery phase corresponds to standard FedAvg rounds

Setting	Method	Efficacy				Efficiency		
		Test Acc.	Forget Acc. ($\Delta \downarrow$)	MIA _[Song] ($\Delta \downarrow$)	MIA _[Yeom] ($\Delta \downarrow$)	Communication Bytes ($\times \uparrow$)	Computation FLOPs ($\times \uparrow$)	Storage Bytes ($\times \uparrow$)
CIFAR-100, IID, ResNet-18, $E=1$	Original	59.9 $\pm$ 0.0	79.8 $\pm$ 0.7	78.2 $\pm$ 0.6	71.4 $\pm$ 0.6	—	—	—
	Retrain	58.3 $\pm$ 0.5	58.0 $\pm$ 0.7	55.6 $\pm$ 0.7	48.8 $\pm$ 0.6	1.62e ¹¹ (1.0x)	1.35e ¹⁵ (1.0x)	4.49e ⁰⁷ (1.0x)
	FedEraser	58.4 $\pm$ 0.5	59.5 (1.5 $\pm$ 0.7)	57.6 (2.0 $\pm$ 0.5)	49.9 (1.1 $\pm$ 1.1)	1.62e ¹¹ (1.0x)	6.75e ¹⁴ (2.0x)	8.98e ¹⁰ (0.0005x)
	PGA	59.1 $\pm$ 0.6	72.8 (14.8 $\pm$ 1.0)	71.8 (16.1 $\pm$ 1.8)	63.0 (14.1 $\pm$ 1.6)	9.86e ⁰⁹ (16.4x)	8.54e ¹³ (15.8x)	4.94e ⁰⁸ (0.0909x)
	MoDe	58.7 $\pm$ 0.5	69.0 (11.1 $\pm$ 0.7)	70.2 (14.6 $\pm$ 2.0)	62.4 (13.5 $\pm$ 0.8)	1.75e ¹⁰ (9.3x)	1.46e ¹⁴ (9.3x)	8.98e ⁰⁷ (0.5x)
	FedAU	59.0 $\pm$ 0.5	74.7 (16.8 $\pm$ 1.0)	76.3 (20.6 $\pm$ 2.1)	61.6 (12.8 $\pm$ 2.4)	2.67e⁰⁹ (60.8x)	2.23e¹³ (60.6x)	4.49e ⁰⁷ (1.0x)
	NoT	58.9 $\pm$ 0.4	70.5 (12.6 $\pm$ 0.8)	69.3 (13.7 $\pm$ 1.9)	63.3 (14.4 $\pm$ 0.8)	5.25e ⁰⁹ (30.9x)	4.39e ¹³ (30.8x)	4.49e ⁰⁷ (1.0x)
	PUF-Special	58.8 $\pm$ 0.3	62.8 (4.9 $\pm$ 1.1)	61.5 (5.9 $\pm$ 1.4)	55.8 (7.0 $\pm$ 1.1)	3.40e ⁰⁹ (47.6x)	2.84e ¹³ (47.5x)	4.49e ⁰⁷ (1.0x)
CIFAR-100, Non-IID, ResNet-18, $E=1$	PUF-Regular	58.8 $\pm$ 0.4	62.4 (4.4$\pm$1.0)	60.8 (5.2$\pm$0.9)	55.5 (6.7$\pm$1.1)	5.10e ⁰⁹ (31.8x)	4.26e ¹³ (31.7x)	4.49e ⁰⁷ (1.0x)
	Original	53.8 $\pm$ 0.0	62.9 $\pm$ 6.9	75.4 $\pm$ 7.0	61.0 $\pm$ 7.8	—	—	—
	Retrain	51.0 $\pm$ 1.4	33.5 $\pm$ 4.5	44.0 $\pm$ 5.5	32.0 $\pm$ 5.2	1.62e ¹¹ (1.0x)	1.35e ¹⁵ (1.0x)	4.49e ⁰⁷ (1.0x)
	FedEraser	51.2 $\pm$ 0.5	35.7 (2.2 $\pm$ 1.6)	49.1 (3.4 $\pm$ 2.0)	47.4 (3.4 $\pm$ 1.5)	1.62e ¹¹ (1.0x)	6.75e ¹⁴ (2.0x)	8.98e ¹⁰ (0.0005x)
	PGA	51.4 $\pm$ 0.6	37.4 (4.3 $\pm$ 4.1)	49.2 (6.4 $\pm$ 4.9)	36.6 (5.3 $\pm$ 5.3)	5.74e ⁰⁹ (28.2x)	5.10e ¹³ (26.5x)	4.94e ⁰⁸ (0.09x)
	MoDe	50.9 $\pm$ 1.1	41.9 (8.4 $\pm$ 3.8)	55.0 (10.6 $\pm$ 4.0)	42.3 (10.3 $\pm$ 3.8)	2.19e ¹⁰ (7.4x)	1.83e ¹⁴ (7.4x)	8.98e ⁰⁷ (0.5x)
	FedAU	51.2 $\pm$ 1.2	38.4 (4.5 $\pm$ 2.2)	51.7 (6.7 $\pm$ 2.5)	38.3 (5.3 $\pm$ 1.9)	1.29e ¹⁰ (12.5x)	1.08e ¹⁴ (12.5x)	4.49e ⁰⁷ (1.0x)
	NoT	51.3 $\pm$ 0.2	43.3 (9.9 $\pm$ 4.4)	55.7 (11.7 $\pm$ 5.7)	44.3 (12.3 $\pm$ 5.5)	1.03e ¹⁰ (15.7x)	8.64e ¹³ (15.6x)	4.49e ⁰⁷ (1.0x)
CIFAR-100, Non-IID, ResNet-18, $E=10$	PUF-Special	52.3 $\pm$ 0.4	36.8 (3.3 $\pm$ 2.2)	47.3 (3.4$\pm$2.8)	35.9 (3.9$\pm$2.6)	4.01e⁰⁹ (40.4x)	3.56e¹³ (38.0x)	4.49e ⁰⁷ (1.0x)
	PUF-Regular	52.2 $\pm$ 0.6	35.4 (2.0$\pm$2.0)	48.8 (4.8 $\pm$ 2.4)	36.4 (4.4 $\pm$ 2.3)	6.44e ⁰⁹ (25.1x)	5.59e ¹³ (24.1x)	4.49e ⁰⁷ (1.0x)
	Original	50.3 $\pm$ 0.0	60.8 $\pm$ 6.6	73.4 $\pm$ 6.6	61.3 $\pm$ 7.5	—	—	—
	Retrain	48.0 $\pm$ 0.8	31.6 $\pm$ 4.5	42.1 $\pm$ 5.2	31.2 $\pm$ 4.9	1.62e ¹¹ (1.0x)	1.35e ¹⁶ (1.0x)	4.49e ⁰⁷ (1.0x)
	PGA	46.3 $\pm$ 1.9	34.7 (3.1 $\pm$ 3.9)	45.3 (3.2 $\pm$ 4.0)	33.5 (2.3 $\pm$ 3.7)	3.01e ¹⁴ (44.9x)	3.35e ¹³ (40.4x)	4.94e ⁰⁸ (0.09x)
	MoDe	46.9 $\pm$ 2.3	35.3 (3.9 $\pm$ 3.6)	45.4 (4.7 $\pm$ 4.1)	34.0 (3.6 $\pm$ 3.3)	2.10e ¹⁰ (7.7x)	1.76e ¹⁵ (7.7x)	8.98e ⁰⁷ (0.5x)
	FedAU	46.8 $\pm$ 2.4	36.8 (4.5 $\pm$ 3.7)	47.1 (2.8 $\pm$ 3.5)	34.7 (2.8 $\pm$ 3.5)	1.13e ¹⁰ (14.3x)	9.45e ¹⁴ (14.3x)	4.49e ⁰⁷ (1.0x)
	NoT	46.7 $\pm$ 2.2	39.7 (8.1 $\pm$ 3.6)	48.9 (6.8 $\pm$ 4.4)	37.2 (6.0 $\pm$ 3.9)	6.22e ⁰⁹ (26.0x)	5.20e ¹⁴ (26.0x)	4.49e ⁰⁷ (1.0x)
CIFAR-10, Non-IID, ResNet-18, $E=1$	PUF-Special	47.8 $\pm$ 1.5	34.0 (2.4 $\pm$ 2.9)	43.7 (1.6$\pm$3.2)	32.4 (1.2$\pm$3.0)	2.90e⁰⁹ (55.8x)	2.67e¹⁴ (50.7x)	4.49e ⁰⁷ (1.0x)
	PUF-Regular	47.6 $\pm$ 1.6	33.3 (1.7$\pm$2.8)	44.5 (2.4 $\pm$ 2.9)	32.7 (1.5 $\pm$ 2.7)	4.85e ⁰⁹ (33.4x)	4.44e ¹⁴ (30.4x)	4.49e ⁰⁷ (1.0x)
	Original	83.7 $\pm$ 0.0	88.6 $\pm$ 3.9	84.4 $\pm$ 5.4	81.4 $\pm$ 6.3	—	—	—
	Retrain	83.5 $\pm$ 1.6	81.1 $\pm$ 8.0	73.1 $\pm$ 13.7	72.5 $\pm$ 10.8	1.62e ¹¹ (1.0x)	1.35e ¹⁵ (1.0x)	4.49e ⁰⁷ (1.0x)
	PGA	84.0 $\pm$ 1.0	84.3 (3.2 $\pm$ 4.9)	78.5 (5.4 $\pm$ 7.7)	77.0 (4.5 $\pm$ 6.0)	9.38e ⁰⁹ (17.3x)	8.14e ¹³ (16.6x)	4.94e ⁰⁸ (0.09x)
	MoDe	83.5 $\pm$ 1.6	82.3 (2.0 $\pm$ 1.7)	72.6 (2.6 $\pm$ 1.8)	71.8 (1.5 $\pm$ 1.0)	2.40e ¹⁰ (6.7x)	2.01e ¹⁴ (6.7x)	8.98e ⁰⁷ (0.5x)
	FedAU	83.8 $\pm$ 1.4	85.2 (4.0 $\pm$ 2.6)	76.9 (4.3 $\pm$ 3.7)	73.2 (2.9 $\pm$ 2.9)	5.17e ⁰⁹ (31.3x)	4.32e ¹³ (31.2x)	4.49e ⁰⁷ (1.0x)
	NoT	83.8 $\pm$ 1.2	84.3 (3.1 $\pm$ 3.0)	78.0 (4.9 $\pm$ 4.5)	75.3 (2.9 $\pm$ 3.4)	1.37e ¹⁰ (11.9x)	1.14e ¹⁴ (11.8x)	4.49e ⁰⁷ (1.0x)
CIFAR-100, Non-IID, MiT-B0, $E=1$	PUF-Special	83.9 $\pm$ 1.2	82.0 (0.9$\pm$1.4)	72.0 (1.4$\pm$1.7)	71.1 (2.1$\pm$2.0)	3.30e⁰⁹ (49.1x)	2.79e¹³ (48.5x)	4.49e ⁰⁷ (1.0x)
	PUF-Regular	83.8 $\pm$ 1.2	82.4 (1.3 $\pm$ 1.6)	72.8 (2.2 $\pm$ 2.0)	71.5 (2.4 $\pm$ 1.8)	5.31e ⁰⁹ (30.5x)	4.49e ¹³ (30.1x)	4.49e ⁰⁷ (1.0x)
	Original	75.0 $\pm$ 0.0	84.3 $\pm$ 5.6	77.0 $\pm$ 8.6	74.0 $\pm$ 6.4	—	—	—
	Retrain	73.3 $\pm$ 0.8	57.8 $\pm$ 5.3	48.5 $\pm$ 5.0	44.6 $\pm$ 3.9	1.19e ¹⁰ (1.0x)	3.82e ¹⁵ (1.0x)	1.33e ⁰⁷ (1.0x)
	PGA	73.6 $\pm$ 0.4	62.2 (4.4 $\pm$ 3.3)	53.6 (5.0 $\pm$ 3.7)	50.5 (6.9 $\pm$ 3.2)	1.68e⁰⁹ (7.1x)	5.70e¹⁴ (6.7x)	1.46e ⁰⁸ (0.09x)
	MoDe	73.4 $\pm$ 0.8	62.9 (5.2 $\pm$ 2.2)	54.7 (6.1 $\pm$ 3.1)	50.3 (5.9 $\pm$ 2.9)	5.55e ⁰⁹ (2.1x)	1.78e ¹⁵ (2.2x)	2.66e ⁰⁷ (0.5x)
	FedAU	73.6 $\pm$ 0.7	63.7 (5.9 $\pm$ 1.9)	55.2 (5.9 $\pm$ 2.3)	50.2 (5.6 $\pm$ 1.3)	3.54e ⁰⁹ (3.4x)	1.13e ¹⁵ (3.4x)	1.33e ⁰⁷ (1.0x)
	NoT	73.4 $\pm$ 0.7	67.9 (10.1 $\pm$ 4.1)	59.4 (10.1 $\pm$ 6.6)	54.0 (9.4 $\pm$ 5.4)	3.70e ⁰⁹ (3.2x)	1.19e ¹⁵ (3.2x)	1.33e ⁰⁷ (1.0x)
CIFAR-100, Non-IID, MiT-B0, $E=1$	PUF-Special	73.7 $\pm$ 0.5	58.2 (0.4$\pm$1.5)	47.2 (1.4$\pm$1.7)	45.1 (0.5$\pm$1.4)	2.65e ⁰⁹ (4.5x)	8.42e ¹⁴ (4.5x)	1.33e ⁰⁷ (1.0x)
	PUF-Regular	73.6 $\pm$ 0.6	59.3 (1.5 $\pm$ 1.7)	48.4 (2.7 $\pm$ 1.8)	45.8 (1.2 $\pm$ 1.3)	3.92e ⁰⁹ (3.0x)	1.26e ¹⁵ (3.0x)	1.33e ⁰⁷ (1.0x)

TABLE III: Performance of PUF and other baselines across different settings for client unlearning. Baseline results are expressed as *mean metric value (mean $\Delta \pm$ standard deviation)*, with Δ in parentheses representing the average absolute difference from the Retrain baseline. Lower Δ corresponds to better unlearning performances. For efficiency metrics, higher $\times$ (reduction over Retrain baseline) are better. The best-performing method is shown in bold. When FedEraser [16] ranks first, it is underlined, as its efficiency limitations make it impractical in realistic scenarios.

involving only retained clients; all methods incur the same per-round costs during recovery, differing only in the number of recovery rounds required to restore model utility. For retraining baselines, the reported costs correspond to training the model from scratch with the remaining clients, i.e., the total FedAvg rounds performed after removing the target clients. We provide a detailed explanation of the overhead calculations for each method in Appendix B (Supplementary Material).

D. Main Results

Client Unlearning. Table III reports the performance of PUF’s best configurations (hyper-parameter tuning in Sec. V-E) and other baselines across multiple datasets (CIFAR-100 and CIFAR-10), data distributions (IID and Non-IID), and model architectures (ResNet-18, MiT-B0) in the single-client unlearning scenario. Results are averaged over ten independent runs, each with a different target client. Across all settings,

PUF attains the lowest or near-lowest Δ values in forget metrics (e.g., Forget Accuracy), meaning it most closely replicates the retraining gold standard, thereby achieving highly effective forgetting.

In **CIFAR-100 (IID, ResNet-18)**, PUF-Special and PUF-Regular reduce Δ Forget Accuracy to below 5%, outperforming MoDe (11.1%), NoT (12.6%), and FedAU (16.8%). While FedAU achieves the highest efficiency overall (communication and computation $\approx 60\times$ lower than retraining), this comes at the cost of poor forgetting performance. Conversely, FedEraser delivers almost perfect forgetting ($\Delta \approx 1.5\%$), but with efficiency close to retraining in communication and computation, and drastically higher storage needs. PUF variants strike a balance, offering sub-5% Δ while maintaining 31–48 $\times$ communication and computation gains.

In **CIFAR-100 (Non-IID, ResNet-18, $E=1$)**, PUF-Regular achieves the best Δ Forget Accuracy (2.0%), ahead of FedAU

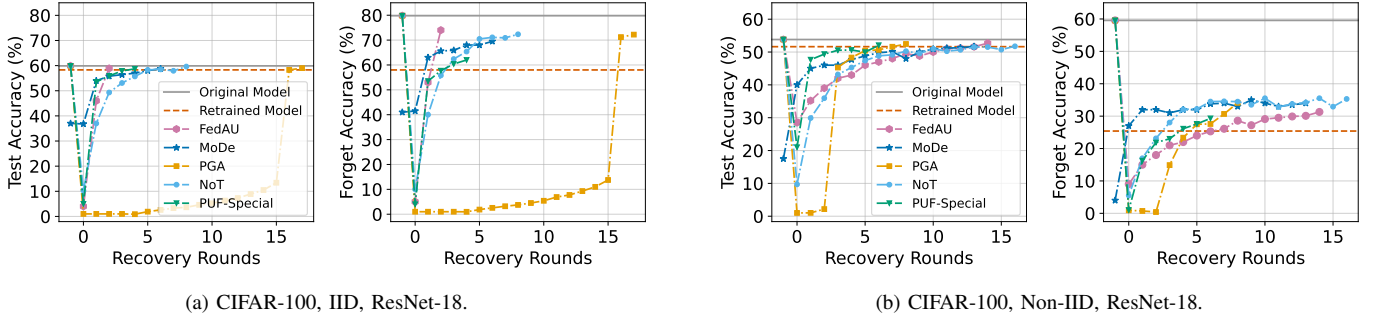


Fig. 4: Evolution of generalization ability (test accuracy) and forgetting effectiveness (forget accuracy) during the recovery phase across three different settings. Each pair of images presents Test Accuracy (**Left**) and Forget Accuracy (**Right**) for a representative client in a specific setting indicated in subcaption as a triple *dataset, data distribution, model architecture*. For our method, the charts only report the performance of PUF-Special for better visualization. For MoDe we do not show the multi-round unlearning phase for clarity.

(4.5%), MoDe (8.4%), and NoT (9.9%), while preserving $25\times$ efficiency gains over retraining. PUF-Special slightly sacrifices forgetting (3.3% Δ) for even higher efficiency ($\approx 40\times$). PGA achieves higher efficiency ($\approx 28\times$) but worse forgetting (4.3% Δ Forget Accuracy).

In **CIFAR-100 (Non-IID, ResNet-18, $E=10$)**, PUF-Regular exhibits the lowest Δ Forget Accuracy (1.7%) with $33\times$ efficiency gains, while PUF-Special offers the highest efficiency ($\approx 56\times$) and competitive forgetting (2.4%). PGA is also efficient ($\approx 44\times$) but less accurate in forgetting (3.1%).

In **CIFAR-10 (Non-IID, ResNet-18)**, PUF-Special reaches the best forgetting performance (Δ Forget Accuracy $< 1\%$) with the highest efficiency ($\approx 49\times$), outperforming all baselines in both dimensions.

Finally, in **CIFAR-100 (Non-IID, MiT-B0)**, PUF-Special attains near-perfect forgetting ($\Delta=0.4\%$) with solid efficiency ($\approx 4.5\times$), while PGA is the most efficient ($\approx 7\times$) but exhibits notably weaker forgetting ($\Delta=4.4\%$). Overall, PUF variants consistently achieve an effective trade-off between efficacy and efficiency.

Performance Consistency During Recovery. Figure 4 illustrates the evolution of Test Accuracy and Forget Accuracy throughout the recovery phase, beginning at recovery round 0, which corresponds to the application of the unlearning routine (either PGA, MoDe, FedAU, NoT, or our proposed solution, PUF). We do not report the evolution of the considered MIAs, as they closely follow the trends of Forget Accuracy. The figure presents results for three different settings, as indicated in the subcaptions. For reference, we also include values for both the Original Model and the Retrained Model. We emphasize the following key aspects for the full and easy understanding of the reported charts:

- 1) The recovery phase concludes when the unlearned model surpasses the Test Accuracy of the retrained model for the first time, where this condition is evaluated on averaged results over multiple runs and target clients to account for variability and reduce the effect of fluctuations.
- 2) A more effective forgetting method should minimize the gap in Forget Accuracy at the end of the recovery phase.
- 3) A better forgetting method should also reduce the discrepancy with the retrained model's Test Accuracy

Method	Efficacy		Efficiency	
	Test Acc.	ASR ($\Delta \downarrow$)	Communication Bytes ($\times \uparrow$)	Computation FLOPs ($\times \uparrow$)
Original	58.63	84.06	—	—
Retrain	58.51	0.42	$1.62e^{11}$ (1.0 $\times$)	$1.35e^{15}$ (1.0 $\times$)
PGA	58.62	4.84 (4.42)	$9.86e^{09}$ (16.4 $\times$)	$8.54e^{13}$ (15.8 $\times$)
MoDe	58.60	4.91 (4.49)	$1.75e^{10}$ (9.3 $\times$)	$1.46e^{14}$ (9.3 $\times$)
FedAU	58.56	5.27 (4.85)	$1.29e^{10}$ (12.5 $\times$)	$1.08e^{14}$ (12.5 $\times$)
NoT	58.55	1.10 (0.68)	$6.44e^{09}$ (25.1 $\times$)	$5.59e^{13}$ (24.1 $\times$)
PUF-Special	58.61	0.52 (0.10)	$3.40e^{09}$ (47.6$\times$)	$2.84e^{13}$ (47.5$\times$)
PUF-Regular	58.65	0.38 (0.04)	$5.10e^{09}$ (31.8 $\times$)	$4.26e^{13}$ (31.7 $\times$)

TABLE IV: Performance of PUF and baselines in the backdoor unlearning setting with one malicious client, considering CIFAR-100, IID data, ResNet-18, and 10 total clients. Backdoor ASR results are reported as *mean value* (Δ), where Δ is the absolute difference from the Retrain baseline. Lower Δ indicates better alignment with retraining-based unlearning.

during the recovery phase, ensuring higher usability of the unlearned model, which serves as the new FL global model for inference during these rounds.

Thus, Figure 4 provides valuable insights into the dynamics of these metrics during the recovery phase. The analysis highlights that PUF produces a superior unlearned model, significantly narrowing the gap with the retrained model earlier in the recovery phase compared to the other baselines. The figure clearly shows that the other baselines fail to achieve accurate forgetting by the end of the recovery phase, as evidenced by their larger gaps in Forget Accuracy. All the methods show an accuracy drop after unlearning. This is an inherent consequence of strategies that operate directly on the global model, as the removal of a client's contributions inevitably results in a temporary degradation of overall performance [13].

Performance with Backdoor Removal. Table IV reports the results for backdoor removal, where ASR is used as the primary metric to assess unlearning effectiveness. Lower ASR indicates better removal of the malicious behavior. Overall, both PUF variants achieve the best performance in terms of ASR, exhibiting the smallest deviation from the retraining baseline and thus providing the most effective backdoor mitigation. At the same time, PUF-Special and PUF-Regular also deliver the strongest improvements in efficiency metrics.

Sample Unlearning. Table V reports the results for sample-level unlearning when 50% of each client's dataset is desig-

Setting	Method	Efficacy				Efficiency		
		Test Acc.	Forget Acc. ($\Delta \downarrow$)	MIA _[Song] ($\Delta \downarrow$)	MIA _[Yeom] ($\Delta \downarrow$)	Communication Bytes ($\times \uparrow$)	Computation FLOPs ($\times \uparrow$)	Storage Bytes ($\times \uparrow$)
CIFAR-100, IID, ResNet-18, $E=1$	Retrain	59.4 $\pm$ 0.4	58.5 $\pm$ 0.5	56.0 $\pm$ 0.6	49.2 $\pm$ 0.7	1.80e ¹¹ (1.0 $\times$)	1.42e ¹⁵ (1.0 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
	FedAU	59.8 $\pm$ 0.3	65.7 (7.2 $\pm$ 1.1)	61.8 (5.8 $\pm$ 0.9)	54.9 (5.7 $\pm$ 0.9)	8.08e⁰⁸ (222.7 $\times$)	7.12e¹² (199.3 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
	NoT	59.7 $\pm$ 0.5	72.4 (13.9 $\pm$ 0.7)	68.1 (12.1 $\pm$ 1.0)	60.7 (11.5 $\pm$ 1.0)	5.65e ⁰⁹ (31.8 $\times$)	4.99e ¹³ (28.5 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
	PUF-Special	60.0 $\pm$ 0.8	64.5 (6.0 $\pm$ 1.1)	60.6 (4.6 $\pm$ 0.9)	53.8 (4.6 $\pm$ 0.9)	1.43e ⁰⁹ (125.8 $\times$)	1.22e ¹³ (116.4 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
	PUF-Regular	60.1 $\pm$ 0.8	64.6 (6.1 $\pm$ 1.2)	60.7 (4.7 $\pm$ 0.9)	53.9 (4.7 $\pm$ 0.9)	2.51e ⁰⁹ (71.6 $\times$)	2.18e ¹³ (65.3 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
CIFAR-100, Non-IID, ResNet-18, $E=1$	Retrain	49.0 $\pm$ 0.6	40.3 $\pm$ 2.4	38.9 $\pm$ 2.0	34.1 $\pm$ 1.8	1.80e ¹¹ (1.0 $\times$)	1.42e ¹⁵ (1.0 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
	FedAU	50.0 $\pm$ 0.2	47.2 (6.9 $\pm$ 1.2)	45.1 (6.2 $\pm$ 1.1)	40.2 (6.1 $\pm$ 1.1)	8.08e⁰⁸ (222.7 $\times$)	7.12e¹² (199.3 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
	NoT	49.6 $\pm$ 0.8	49.0 (8.8 $\pm$ 1.4)	47.0 (8.1 $\pm$ 1.3)	41.8 (7.7 $\pm$ 1.2)	6.46e ⁰⁹ (27.9 $\times$)	5.70e ¹³ (24.9 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
	PUF-Special	50.1 $\pm$ 1.0	42.5 (2.2 $\pm$ 1.2)	40.9 (2.0 $\pm$ 1.1)	36.3 (2.2 $\pm$ 1.1)	8.98e ⁰⁸ (200.5 $\times$)	7.50e ¹² (189.3 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
	PUF-Regular	50.2 $\pm$ 1.0	41.3 (4.3 $\pm$ 1.3)	40.1 (4.0 $\pm$ 1.2)	35.2 (4.1 $\pm$ 1.2)	1.71e ⁰⁹ (105.5 $\times$)	1.46e ¹³ (97.1 $\times$)	4.49e ⁰⁷ (1.0 $\times$)

TABLE V: Performance of PUF and other baselines across different settings for sample unlearning when half of the dataset is the forget data. For efficacy metrics, baseline results are expressed as *mean metric value (mean $\Delta \pm$ standard deviation)*, with Δ in parentheses representing the average absolute difference from the Retrain baseline. Lower Δ corresponds to better unlearning performances. For efficiency metrics, higher $\times$ (reduction over Retrain baseline) are better. The best-performing method is shown in bold.

Setting	Method	Efficacy				Efficiency		
		Test Acc.	Forget Acc. ($\Delta \downarrow$)	MIA _[Song] ($\Delta \downarrow$)	MIA _[Yeom] ($\Delta \downarrow$)	Communication Bytes ($\times \uparrow$)	Computation FLOPs ($\times \uparrow$)	Storage Bytes ($\times \uparrow$)
Multiple Unlearning, CIFAR-100, Non-IID, ResNet-18, $E=1$	Original	53.8 $\pm$ 0.0	63.1 $\pm$ 6.8	76.0 $\pm$ 7.6	61.1 $\pm$ 8.3	—	—	—
	Retrain	47.3 $\pm$ 2.1	30.1 $\pm$ 3.3	39.7 $\pm$ 3.9	27.9 $\pm$ 3.1	1.44e ¹¹ (1.0 $\times$)	1.20e ¹⁵ (1.0 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
	FedAU	48.3 $\pm$ 2.3	35.6 (5.7 $\pm$ 2.4)	45.6 (6.4 $\pm$ 3.6)	33.6 (5.7 $\pm$ 4.1)	7.90e ⁰⁹ (18.2 $\times$)	6.60e ¹³ (18.2 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
	NoT	48.1 $\pm$ 2.2	39.2 (9.1 $\pm$ 2.8)	50.5 (11.3 $\pm$ 3.4)	41.8 (13.9 $\pm$ 4.5)	4.74e ⁰⁹ (30.4 $\times$)	3.96e ¹³ (30.3 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
	PUF-Special	48.5 $\pm$ 2.1	34.1 (4.0 $\pm$ 3.7)	46.1 (6.6 $\pm$ 3.5)	33.5 (5.6 $\pm$ 4.2)	2.48e⁰⁹ (58.1 $\times$)	2.07e¹³ (58.0 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
	PUF-Regular	47.9 $\pm$ 2.3	31.2 (2.9 $\pm$ 1.9)	42.9 (3.9 $\pm$ 2.9)	31.4 (3.8 $\pm$ 3.1)	7.54e ⁰⁹ (19.1 $\times$)	5.55e ¹³ (21.6 $\times$)	4.49e ⁰⁷ (1.0 $\times$)
Multiple Unlearning, CIFAR-100, Non-IID, MiT-B0, $E=1$	Original	75.0 $\pm$ 0.0	84.6 $\pm$ 5.3	76.8 $\pm$ 9.0	73.8 $\pm$ 1.4	—	—	—
	Retrain	70.9 $\pm$ 1.3	54.3 $\pm$ 3.5	45.4 $\pm$ 3.6	43.5 $\pm$ 1.6	1.06e ¹⁰ (1.0 $\times$)	3.40e ¹⁵ (1.0 $\times$)	1.33e ⁰⁷ (1.0 $\times$)
	FedAU	71.4 $\pm$ 0.9	58.9 (4.6 $\pm$ 2.3)	50.1 (4.7 $\pm$ 2.3)	47.2 (3.7 $\pm$ 1.9)	3.19e ⁰⁹ (3.3 $\times$)	1.02e ¹⁵ (3.3 $\times$)	1.33e ⁰⁷ (1.0 $\times$)
	NoT	71.4 $\pm$ 0.9	69.9 (15.6 $\pm$ 6.3)	62.5 (16.6 $\pm$ 8.9)	59.0 (15.5 $\pm$ 7.5)	6.80e⁰⁸ (15.6 $\times$)	2.18e¹⁴ (15.6 $\times$)	1.33e ⁰⁷ (1.0 $\times$)
	PUF-Special	71.3 $\pm$ 1.4	60.6 (6.3 $\pm$ 2.1)	53.0 (7.3 $\pm$ 2.7)	50.5 (7.0 $\pm$ 1.8)	1.12e ⁰⁹ (9.5 $\times$)	3.57e ¹⁴ (9.5 $\times$)	1.33e ⁰⁷ (1.0 $\times$)
	PUF-Regular	70.9 $\pm$ 1.5	56.6 (2.3 $\pm$ 1.1)	48.7 (2.6 $\pm$ 2.3)	45.6 (2.1 $\pm$ 1.1)	3.93e ⁰⁹ (2.7 $\times$)	1.17e ¹⁵ (2.9 $\times$)	1.33e ⁰⁷ (1.0 $\times$)

TABLE VI: Performance of PUF and other baselines across different settings for multiple unlearning when two clients request forgetting. For efficacy metrics, baseline results are expressed as *mean metric value (mean $\Delta \pm$ standard deviation)*, with Δ in parentheses representing the average absolute difference from the Retrain baseline. Lower Δ corresponds to better unlearning performances. For efficiency metrics, higher $\times$ (reduction over Retrain baseline) are better. The best-performing method is shown in bold.

nated as forget data. Across both IID and non-IID CIFAR-100 settings, PUF consistently delivers the smallest Δ Forget Accuracy, confirming its superior forgetting effectiveness over all baselines. In the IID setting, PUF-Special achieves a Δ Forget Accuracy of only 6.0%, outperforming FedAU (7.2%) and substantially better than NoT (13.9%). Similar trends are observed in the auxiliary membership inference metrics (MIA_{Song} and MIA_{Yeom}), where PUF-Special reduces the Δ to 4.6%, the lowest among all methods. PUF-Regular matches this forgetting effectiveness closely (Δ Forget Accuracy 6.1%), still outperforming FedAU and clearly surpassing NoT. In the non-IID scenario, PUF-Special maintains a Δ Forget Accuracy of only 2.2%, significantly ahead of FedAU (6.9%) and NoT (8.8%). Also PUF-Regular, at 4.3%, outperforms both baselines by a clear margin. These results indicate that PUF’s update-scaling strategy remains effective under higher data heterogeneity, where other methods experience substantial degradation in forgetting performance. Concerning efficiency, both PUF variants achieve large reductions in cumulative communication and computation costs compared to Retrain, up to 200.5 $\times$ and 189.3 $\times$ for PUF-Special in the non-IID setting. While FedAU attains the highest cost reduction (222.7 $\times$ communication, 199.3 $\times$ computation), this comes at the expense of weaker forgetting (e.g., Δ Forget Accuracy 6.9% in non-IID vs. 2.2% for PUF-Special). NoT, conversely,

exhibits both lower efficiency (e.g., 27.9 $\times$ communication gain in non-IID) and weaker forgetting effectiveness. Overall, the results confirm that PUF offers a favorable balance between forgetting accuracy and efficiency in sample unlearning, matching or exceeding the best efficiency reductions while preserving retrain-level forgetting performance.

Multiple Unlearning. Table VI presents the performance on federated CIFAR-100 (non-IID) using a ResNet-18 or an MiT-B0 architecture, where a pair of randomly chosen clients was designated as target clients (averaged over five independent experiments). We do not include PGA as a baseline because it is not directly extensible to unlearning multiple tasks. This limitation arises from its reliance on a reference model, which is obtained by removing the last stored model update of the target client. As a result, the mutual dependency of unlearning updates prevents its straightforward adaptation to multiple requests.

For ResNet-18, PUF-Regular achieves the most faithful replication of retraining, with a Forget Accuracy gap of just 2.9%, accompanied by equally low deviations in both MIA metrics. PUF-Special follows closely, with only a modest increase in the forgetting gap (4.0%) but delivering the best efficiency gains of all methods, cutting communication and computation costs by more than a factor of 58. By contrast, FedAU maintains good forgetting performance (5.7% gap) and

Setting	Method	Efficacy			Efficiency	
		Test Acc.	Forget Acc. ($\Delta \downarrow$)	Forget Loss ($\Delta \downarrow$)	Communication Bytes ($\times \uparrow$)	Computation FLOPs ($\times \uparrow$)
ProstateMRI, U-NET	Original	86.5 $\pm$ 0.0	88.9 $\pm$ 4.6	0.08 $\pm$ 0.05	—	—
	Retrain	82.7 $\pm$ 1.7	73.3 $\pm$ 7.2	0.22 $\pm$ 0.10	2.74e ¹¹ (1.0 $\times$)	5.49e ¹⁶ (1.0 $\times$)
	PGA	82.9 $\pm$ 0.4	77.0 (3.8 $\pm$ 2.3)	0.18 (0.06 $\pm$ 0.02)	2.69e ¹⁰ (10.2 $\times$)	5.44e ¹⁵ (10.1 $\times$)
	NoT	83.4 $\pm$ 0.7	77.4 (3.9 $\pm$ 2.4)	0.17 (0.06 $\pm$ 0.02)	2.63e ¹⁰ (10.4 $\times$)	5.27e ¹⁵ (10.4 $\times$)
	PUF-Special	83.1 $\pm$ 0.4	76.7 (3.4$\pm$2.3)	0.17 (0.05$\pm$0.02)	2.30e¹⁰ (11.9$\times$)	4.62e¹⁵ (11.9$\times$)

TABLE VII: Performance of PUF and other baselines across for ProstateMRI segmentation task. For efficacy metrics, Lower Δ corresponds to better unlearning performances. For efficiency metrics, higher $\times$ (reduction over Retrain baseline) are better. The best-performing method is shown in bold.

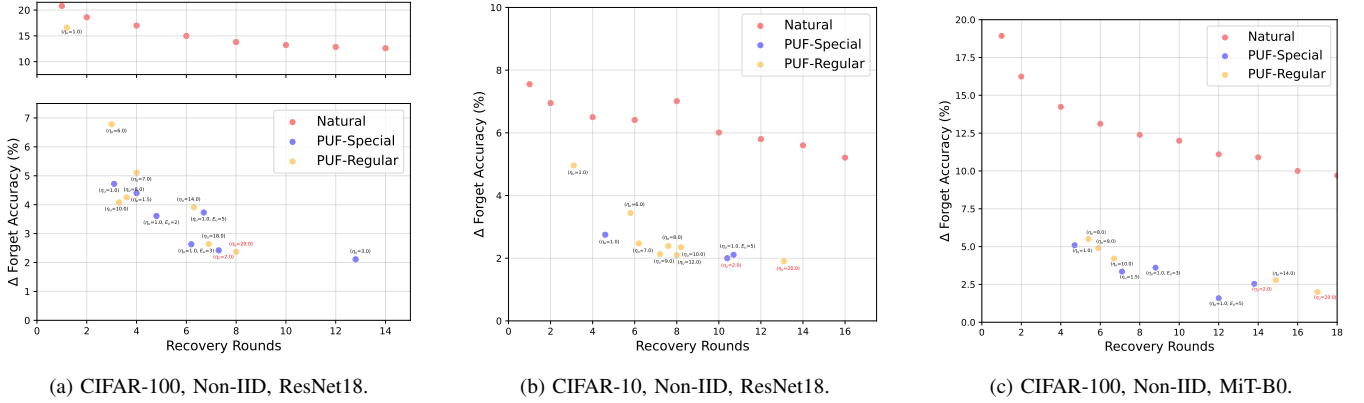


Fig. 5: **Performance of PUF with varying hyper-parameters** (η_u , E_u). When E_u is omitted, it is set to 1. **X-axis**: number of recovery rounds required to match the retrained model’s test accuracy. **Y-axis**: gap in forget accuracy compared to the retrained model. Points closer to the origin indicate better performance. The experimental setting is indicated in subcaption as a triple *dataset, data distribution, model architecture*. The *Natural* baseline reports results for a naive strategy of fine tuning the global model without the participation of the unlearning client, no explicit unlearning is performed.

offers notable savings (18 $\times$), but still falls short of PUF in both accuracy preservation and privacy metrics. NoT shows clear signs of incomplete forgetting, with a gap exceeding 9%, which is also reflected in higher MIA scores.

The pattern is consistent for MiT-B0. PUF-Regular exhibits the smallest gap to retraining (2.3%), outperforming FedAU (4.6%) by nearly half and dramatically surpassing NoT, which exhibits a large 15.6% gap. PUF-Special maintains solid forgetting (6.3%) while achieving higher efficiency than FedAU, but slightly less than NoT, which remains the most communication-efficient method here (15.6 $\times$) at the cost of very weak forgetting.

Task Agnosticism. As highlighted in Sections II and IV, the design of PUF ensures that the unlearning phase remains task-agnostic, as both the unlearning and remaining clients follow the standard training procedure without any modifications to their local learning routines. Table VII reports the results for the ProstateMRI experiments, which focus on a segmentation task rather than classification. FedAU is not applicable to segmentation tasks like Res-UNet, as it requires a separate classification head, which such architectures lack. MoDe’s memory-guidance phase is also not clearly adaptable to non-classification tasks, with no instructions in the original paper. PUF outperforms all baselines in terms of both recovery efficiency (i.e., shorter recovery phase) and forgetting efficacy. As discussed in Section V-E, PUF can be further tuned for improved forgetting performance by increasing the scaling

factor η_u , albeit at the cost of a longer recovery phase. Notably, NoT does not offer this tuning flexibility.

E. Hyper-parameter Tuning of PUF

Figure 5 compares the unlearning performance (Δ Forget Accuracy on the Y-axis, recovery rounds on the X-axis) of various configurations of PUF-Special and PUF-Regular with the best configurations of NoT and PGA, and the Natural baseline. The analysis also includes configurations where PUF employs more local epochs (denoted as E_u in Figure 5; when omitted, $E_u = 1$). Points closer to the origin of the axes indicate better performance. For this analysis, we focused exclusively on the Δ Forget Accuracy metric, as the MIAs metrics align closely with this metric (as emerges from tabular results). We highlight in red the configurations we found to be the best and used them in the Experimental Results section.

Notably, increasing local epochs ($E_u > 1$) for PUF reduces Δ Forget Accuracy but requires more recovery rounds compared to $E_u = 1$ with the same η_u . Interestingly, increasing the scaling factor η_u while keeping $E_u = 1$ achieves similar improvements without additional local computational overhead for target clients, making it more practical in resource-constrained settings. Both PUF-Regular and PUF-Special achieve high effectiveness by simply tuning the scaling factor (η_u) while maintaining $E_u = 1$. As shown in Figure 5, PUF demonstrates low sensitivity to the η_u hyper-parameter, resulting in a cluster of well-performing configurations.

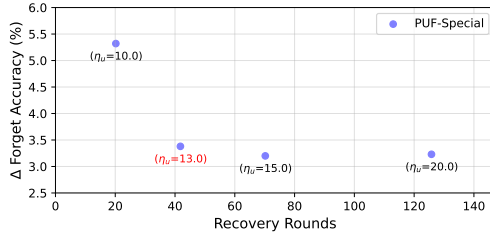


Fig. 6: Performance of PUF with varying hyper-parameter η_u for the ProstateMRI setting.

Notably, the optimal value of the scaling factor η_u differs between PUF-Special and PUF-Regular. This is not merely empirical, but can be explained by the aggregation dynamics of FedAvg. In PUF-Special, the unlearning update is applied in isolation, and thus moderate scaling is sufficient. In contrast, in PUF-Regular, the pseudo-gradient of the target client is averaged together with updates from other participating clients, leading to a dilution of its effect proportional to the number of clients per round. As a result, larger values of η_u are required to compensate for this averaging effect and achieve comparable unlearning performance.

In contrast, we observe that other baseline methods require multiple hyper-parameter to tune. For example, PGA necessitates adjusting the local learning rate, local epochs, an early stopping threshold, and gradient-clipping threshold, while also assuming full client participation and stateful clients. MoDE involves tuning the number of memory-guidance and degradation rounds, separate learning rates for each, and a degradation parameter controlling momentum. FedAU needs to tune the weight to assign to the auxiliary module when merging it with the original model, as well as the number of epochs to train the module. On the other hand, NoT does not require specific hyperparameter tuning but fails to achieve significant forgetting, as shown in Table III. This highlights PUF’s trade-off between ease of integration, ease of hyperparameter optimization, and unlearning performance compared to other baselines.

Figure 6 shows the hyper-parameter tuning of PUF for the ProstateMRI dataset, on segmentation data and real-world feature heterogeneity. Also in this setting, PUF exhibits low sensitivity to its main hyper-parameter.

VI. CONCLUSION

In this paper, we introduced PUF, a novel FU method that leverages negated pseudo-gradients to enable clients to exercise their right to be forgotten. Aligned with the design principles of FL, PUF does not require storing any processed client information and relies only on ephemeral updates, even during unlearning. It integrates seamlessly with standard FedAvg, by requiring no significant modifications or additional computational/communication overhead. As a result, PUF is both task-agnostic and capable of handling multiple unlearning requests concurrently, which are two critical aspects that most existing literature does not consider. Experimental results on two classification datasets and a segmentation task, across varying degrees of data heterogeneity and different model

architectures, are reported to show the general applicability of the proposed approach. PUF has clearly demonstrated to be more efficient in recovering expected performance and more accurate in inducing forgetting of requested data compared to state-of-the-art baselines.

REFERENCES

- [1] R. Arshad and M. R. Asghar, “Characterisation and quantification of user privacy: Key challenges, regulations, and future directions,” *IEEE Communications Surveys & Tutorials*, pp. 1–1, 2024.
- [2] European Parliament and Council of the European Union. (2016) Regulation (EU) 2016/679 of the European Parliament and of the Council. [Online]. Available: <https://data.europa.eu/eli/reg/2016/679/oj>
- [3] “Regulation (EU) 2024/1183 of the European Parliament and of the Council of 5 June 2024 on European digital identity wallets,” 2024, accessed: 2024-11-20. [Online]. Available: <https://eur-lex.europa.eu/eli/reg/2024/1183/oj>
- [4] State of California Department of Justice, “California consumer privacy act (ccpa),” URL <https://oag.ca.gov/privacy/ccpa>, 2018, accessed on October 2023.
- [5] Y. Cao and J. Yang, “Towards Making Systems Forget with Machine Unlearning,” in *2015 IEEE Symposium on Security and Privacy*, 2015, pp. 463–480.
- [6] T. Shaik, X. Tao, H. Xie, L. Li, X. Zhu, and Q. Li, “Exploring the Landscape of Machine Unlearning: A Comprehensive Survey and Taxonomy,” *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–21, 2024.
- [7] J. Liu, P. Ram, Y. Yao, G. Liu, Y. Liu, P. SHARMA, S. Liu *et al.*, “Model sparsity can simplify machine unlearning,” *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [8] V. S. Chundawat, A. K. Tarun, M. Mandal, and M. Kankanhalli, “Can bad teaching induce forgetting? unlearning in deep networks using an incompetent teacher,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 6, 2023, pp. 7210–7217.
- [9] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, “Communication-efficient learning of deep networks from decentralized data,” in *Artificial intelligence and statistics*. PMLR, 2017, pp. 1273–1282.
- [10] K. Bonawitz, H. Eichner, W. Grieskamp, D. Huba, A. Ingerman, V. Ivanov, C. Kiddon, J. Konecny, S. Mazzocchi, H. B. McMahan *et al.*, “Towards federated learning at scale: System design,” *arXiv preprint arXiv:1902.01046*, 2019.
- [11] P. Bellavista, L. Foschini, and A. Mora, “Decentralised Learning in Federated Deployment Environments: A System-Level Survey,” *ACM Computing Surveys (CSUR)*, vol. 54, no. 1, pp. 1–38, 2021.
- [12] O. Marfoq, G. Neglia, R. Vidal, and L. Kameni, “Personalized federated learning through local memorization,” in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 15 070–15 092. [Online]. Available: <https://proceedings.mlr.press/v162/marfoq22a.html>
- [13] N. Romandini, A. Mora, C. Mazzocca, R. Montanari, and P. Bellavista, “Federated Unlearning: A Survey on Methods, Design Guidelines, and Evaluation Metrics,” *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–21, 2024.
- [14] Y. Zhao, J. Yang, Y. Tao, L. Wang, X. Li, and D. Niyato, “A survey of federated unlearning: A taxonomy, challenges and future directions,” *arXiv preprint arXiv:2310.19218*, 2023.
- [15] Y. Liu, L. Xu, X. Yuan, C. Wang, and B. Li, “The right to be forgotten in federated learning: An efficient realization with rapid retraining,” in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 1749–1758.
- [16] G. Liu, X. Ma, Y. Yang, C. Wang, and J. Liu, “Federaser: Enabling efficient client-level data removal from federated learning models,” in *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQoS)*, 2021, pp. 1–10.
- [17] C. Wu, S. Zhu, and P. Mitra, “Federated unlearning with knowledge distillation,” *arXiv preprint arXiv:2201.09441*, 2022.
- [18] C. Wu, S. Zhu, P. Mitra, and W. Wang, “Unlearning backdoor attacks in federated learning,” in *2024 IEEE Conference on Communications and Network Security (CNS)*. IEEE, 2024, pp. 1–9.
- [19] A. Halimi, S. Kadhe, A. Rawat, and N. Baracaldo, “Federated unlearning: How to efficiently erase a client in fl?” *arXiv preprint arXiv:2207.05521*, 2022.

- [20] Y. Zhao, P. Wang, H. Qi, J. Huang, Z. Wei, and Q. Zhang, "Federated unlearning with momentum degradation," *IEEE Internet of Things Journal*, vol. 11, no. 5, pp. 8860–8870, 2023.
- [21] H. Gu, G. Zhu, J. Zhang, X. Zhao, Y. Han, L. Fan, and Q. Yang, "Unlearning during learning: an efficient federated machine unlearning method," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, 2024, pp. 4035–4043.
- [22] Y. H. Khalil, L. Brunswic, S. Lamghari, X. Li, M. Beitollahi, and X. Chen, "NoT: Federated Unlearning via Weight Negation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025.
- [23] P. Kairouz, H. B. McMahan, B. Avent, A. Bellet, M. Bennis, A. N. Bhagoji, K. Bonawitz, Z. Charles, G. Cormode, R. Cummings *et al.*, "Advances and open problems in federated learning," *Foundations and trends® in machine learning*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [24] G. K. Nayak, K. R. Mopuri, and A. Chakraborty, "Effectiveness of Arbitrary Transfer Sets for Data-free Knowledge Distillation," in *Proc. of the IEEE/CVF Winter Conference on Applications of Computer Vision*, 2021, pp. 1430–1438.
- [25] A. Mora, I. Tenison, P. Bellavista, and I. Rish, "Knowledge distillation in federated learning: a practical guide," in *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence*, ser. IJCAI '24, 2024. [Online]. Available: <https://doi.org/10.24963/ijcai.2024/905>
- [26] A. Mora, L. Dominici, and P. Bellavista, "Fedunran: On-device federated unlearning via random labels," in *2024 IEEE International Conference on Big Data (BigData)*. IEEE, 2024, pp. 7955–7960.
- [27] S. J. Reddi, Z. Charles, M. Zaheer, Z. Garrett, K. Rush, J. Konečný, S. Kumar, and H. B. McMahan, "Adaptive federated optimization," in *9th International Conference on Learning Representations, ICLR 2021, Virtual Event, Austria, May 3-7, 2021*. OpenReview.net, 2021. [Online]. Available: <https://openreview.net/forum?id=LkFG3IB13U5>
- [28] J. Wang, S. Guo, X. Xie, and H. Qi, "Federated Unlearning via Class-Discriminative Pruning," in *Proceedings of the ACM Web Conference 2022*, ser. WWW '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 622–632.
- [29] H. Gu, W. Ong, C. S. Chan, and L. Fan, "Ferrari: federated feature unlearning via optimizing feature sensitivity," *Advances in Neural Information Processing Systems*, vol. 37, pp. 24 150–24 180, 2025.
- [30] Z. Liu, Y. Jiang, J. Shen, M. Peng, K.-Y. Lam, X. Yuan, and X. Liu, "A survey on federated unlearning: Challenges, methods, and future directions," *ACM Computing Surveys*, vol. 57, no. 1, pp. 1–38, 2024.
- [31] R. Shokri, M. Stronati, C. Song, and V. Shmatikov, "Membership inference attacks against machine learning models," in *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2017, pp. 3–18.
- [32] H. Hu, Z. Salicic, G. Dobbie, J. Chen, L. Sun, and X. Zhang, "Membership inference via backdooring," in *The 31st International Joint Conference on Artificial Intelligence (IJCAI-22)*, 2022.
- [33] M. A. Rahman, T. Rahman, R. Laganière, N. Mohammed, and Y. Wang, "Membership inference attack against differentially private deep learning model," *Transactions on Data Privacy*, vol. 11, no. 1, pp. 61–79, 2018.
- [34] C. Wu, S. Zhu, and P. Mitra, "Federated unlearning with knowledge distillation," *arXiv preprint arXiv:2201.09441*, 2022.
- [35] Z. Liu, H. Ye, C. Chen, Y. Zheng, and K.-Y. Lam, "Threats, attacks, and defenses in machine unlearning: A survey," *IEEE Open Journal of the Computer Society*, vol. 6, pp. 413–425, 2025.
- [36] P. Blanchard, R. Guerraoui, J. Stainer *et al.*, "Machine learning with adversaries: Byzantine tolerant gradient descent," in *Advances in Neural Information Processing Systems*, 2017, pp. 119–129.
- [37] F. Colosimo and F. De Rango, "Median-krum: A joint distance-statistical based byzantine-robust algorithm in federated learning," in *Proceedings of the Int'l ACM Symposium on Mobility Management and Wireless Access*, 2023, pp. 61–68.
- [38] A. Krizhevsky, "Learning multiple layers of features from tiny images," Tech. Rep., 2009.
- [39] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," *arXiv preprint arXiv:1909.06335*, 2019.
- [40] Q. Liu, Q. Dou, L. Yu, and P. A. Heng, "Ms-net: Multi-site network for improving prostate segmentation with heterogeneous mri data," *IEEE Transactions on Medical Imaging*, 2020.
- [41] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. of IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [42] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, "Segformer: Simple and efficient design for semantic segmentation with transformers," *Advances in Neural Information Processing Systems*, vol. 34, pp. 12 077–12 090, 2021.
- [43] L. Yu, X. Yang, H. Chen, J. Qin, and P. A. Heng, "Volumetric convnets with mixed residual connections for automated prostate segmentation from 3d mr images," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 31, no. 1, 2017.
- [44] J. Kim, G. Kim, and B. Han, "Multi-level branched regularization for federated learning," in *International Conference on Machine Learning*. PMLR, 2022, pp. 11 058–11 073.
- [45] T. Zhou and E. Konukoglu, "Fedfa: Federated feature augmentation," *arXiv preprint arXiv:2301.12995*, 2023.
- [46] F. Milletari, N. Navab, and S.-A. Ahmadi, "V-net: Fully convolutional neural networks for volumetric medical image segmentation," in *2016 fourth international conference on 3D vision (3DV)*. Ieee, 2016, pp. 565–571.
- [47] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [48] L. Song and P. Mittal, "Systematic evaluation of privacy risks of machine learning models," in *30th USENIX Security Symposium (USENIX Security 21)*, 2021, pp. 2615–2632.
- [49] S. Yeom, I. Giacomelli, M. Fredrikson, and S. Jha, "Privacy risk in machine learning: Analyzing the connection to overfitting," in *2018 IEEE 31st computer security foundations symposium (CSF)*. IEEE, 2018, pp. 268–282.



Alessio Mora received his Ph.D. in Computer Science and Engineering in 2023 from the University of Bologna, Bologna, Italy. He is currently an Assistant Professor at the University of Bologna. His research interests include decentralized learning, with a particular focus on federated learning and edge intelligence. He has served on the program committees of top-tier AI conferences, such as IJCAI.



Carlo Mazzocca received his Ph.D. in Computer Science and Engineering from the University of Bologna, Bologna, Italy, in 2024. Currently, he is an Assistant Professor in Tenure Track at the University of Salerno, Italy. His research primarily focuses on digital identity, security, and privacy of Federated Learning. He serves as Consulting Area Editor for IEEE Transactions on Information Forensics and Security. He has served on the program committees of top-tier cybersecurity conferences, including USENIX Security, NDSS, and CCS.



Rebecca Montanari is a Full Professor at the University of Bologna, Italy, focusing on blockchain, information security, and middleware solutions for mobile and IoT services. She has collaborated with leading research centers in the semantic web and policy management, including Nokia Research Center Cambridge, MIT, Imperial College London, and IHMC Pensacola. She has served as coordinator and participant in numerous ICT research projects across Europe and internationally, with funding from Italian ministries and regional programs.



Paolo Bellavista is professor of distributed and mobile systems at the University of Bologna, where he leads the Mobile Middleware research group (<https://site.unibo.it/middleware/en>). His research interests include middleware for mobile computing, dynamic QoS management in the cloud continuum, infrastructures for big data processing in industrial environments, digital twins for industrial automation and smart cities, and performance optimization in wide-scale and latency-sensitive deployment environments. He is Associate EIC of IEEE COMST and serves on the Editorial Boards of several IEEE/ACM/Elsevier international journals.